

Pixels to Predictions: Compact Vision–Language Reasoning under a 5 M Trainable-Parameter Cap

Adam Weidman

New York University, Tandon
CS-GY 6953 / ECE-GY 7123
Deep Learning, Spring 2026
afw8937@nyu.edu

Abstract

We fine-tune SmolVLM-500M-Instruct on a 3,109-example multimodal scientific multiple-choice dataset (ScienceQA-style) under strict academic constraints: at most 5 M trainable parameters, no external data, and offline inference only. Our final pipeline combines (i) low-rank adaptation (Hu et al., 2022) on all seven Llama target modules of the language tower with $r = 8$, $\alpha/r = 1.0$, and per-example choice-permutation augmentation, with (ii) a perceptual-hash retrieval overlay that direct-copies answers from training neighbors when image hash, question text, and choice set all match. Across roughly thirty experiments we ran twelve targeted ablations (LoRA target set, rank, learning rate, alpha, augmentation, image preprocessing, connector unfreezing, DoRA (Liu et al., 2024), training-on-validation, chat-template prompting, retrieval overlays, per-letter calibration). Our best public-leaderboard score is **0.861**, from a higher-LR LoRA variant (iter4-vd, $\text{lr}=4 \times 10^{-4}$) with a retrieval overlay applied at $h = 4$, $q = 0.85$. Plain V_β reached 0.86 on public — close but slightly below. Two “higher-expected” candidates from train∪val merging (finfit, finfit + retrieval) looked stronger on local validation but *underperformed* on public LB (0.823 and 0.849 respectively), so a clean negative result also lands in this report: local validation accuracy was an unreliable selection signal once adapters started saturating it.

1 Introduction

This work targets the *Pixels to Predictions* Kaggle competition: predict a 0-indexed answer for each of 1,008 test rows where each row pairs a scientific image with a question, an optional hint, an optional lecture context, and a JSON list of multiple-choice options. The competition mandates the SmolVLM-500M-Instruct backbone (Laurençon et al., 2024), restricts compute to free Colab/Kaggle

tiers, forbids external data, requires offline inference, and caps trainable parameters at 5 M. Our contributions are:

- A LoRA recipe (V_β) that lifts the zero-shot SmolVLM floor from 0.558 to 0.8073 validation / 0.86 public LB at 4.78 M trainable parameters.
- A perceptual-hash retrieval overlay that adds roughly +2.6 pp validation accuracy at zero training cost by direct-copying answers from nearest train neighbors. Layered on top of a higher-LR variant (iter4-vd), it produces our top public-LB result of **0.861**; layered on top of a train∪val final-fit, it does not transfer.
- Twelve targeted ablations across target-module choice, rank, learning rate, augmentation, image preprocessing, connector unfreezing, adapter scheme (LoRA vs. DoRA), training-on-validation, chat templating, retrieval thresholds, and per-letter calibration.
- Two “best expected” submissions (finfit and finfit + retrieval) underperformed plain V_β and iter4-vd + retrieval on public, despite scoring higher on val. We treat this as evidence that local validation accuracy can be a misleading selection signal for adapters of this size on a 1,048-row hold-out.

2 Task Description

2.1 Problem Setup

Each example i provides an image I_i , a question text q_i , a set of choices $C_i = \{c_1, \dots, c_{n_i}\}$ with $n_i \in \{2, 3, 4, 5\}$, and optional pedagogical context (hint, lecture, subject, topic, skill). The label $y_i \in \{0, \dots, n_i - 1\}$ is provided for train (3,109) and validation (1,048) and withheld for test (1,008). The competition splits the test labels into a public subset for the live leaderboard and a private subset for the final ranking.

2.2 Evaluation Metric

The metric is plain classification accuracy:

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{y}_i = y_i]. \quad (1)$$

A submission is a CSV with exactly the columns `id` and `answer`; rows with missing ids, extra columns, or out-of-range answers are scored as invalid.

3 Methodology

3.1 Base Model

SmolVLM-500M-Instruct (Laurençon et al., 2024) is an Idefics3-architecture vision-language model: a SigLIP vision tower feeds a learned cross-modal projection (Idefics3SimpleMLP) into a 32-layer SmolLM2 (Llama-style) text decoder with hidden dimension 960 and intermediate dimension 2,560. The total parameter count is 507,482,304. We freeze the entire model and inject trainable parameters via PEFT adapters under the 5 M cap.

3.2 Choice-Letter Log-Likelihood Scoring

Following Wiegrefe et al. (2023) and Tsvilodub et al. (2024), we score multiple-choice answers by single-letter argmax over the next-token log-probability at the position immediately following `Answer:`. Concretely, for each row we compute $\log p_\theta(t_k \mid \text{prompt})$ for $t_k \in \{“ A”, “ B”, “ C”, “ D”, “ E”\}$ and predict $\hat{y} = \arg \max_k \log p_\theta(t_k)$. We verified offline that each candidate maps to a single tokenizer id, so the masked cross-entropy target is exactly one position. We considered full-string scoring of choice text but ruled it out on the basis of Wiegrefe et al. (2023), who report it can hurt instruction-tuned models.

3.3 LoRA Fine-Tuning (V_β)

The V_β recipe applies LoRA (Hu et al., 2022) on all seven Llama target modules in the language tower (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) across all 32 SmolLM2 layers, with $r = 8$, $\alpha = 8$ (so $\alpha/r = 1.0$), dropout 0.05, vision tower frozen, no DoRA, no rsLoRA. Trainable parameter count: **4,784,128**, well under the 5 M cap. Loss is causal-LM cross-entropy with all positions masked to `-100` except the answer-letter position. Optimizer is AdamW (Loshchilov and Hutter, 2019) with learning rate 2×10^{-4} , cosine decay with 5% linear warmup, no weight decay, and bfloat16 autocast on a single L40S. We

train 5 epochs at batch size 4 with gradient accumulation of 2 (effective batch size 8); a per-epoch checkpoint preserves the best-validation adapter.

3.4 Choice-Permutation Augmentation

Each `__getitem__` returns the same training row with a random permutation of choices and the answer index updated accordingly. For a 4-choice problem this yields $4! = 24$ distinct orderings observed across epochs. The intent is twofold: (a) eliminate position-bias memorization (otherwise the model can shortcut via the empirical letter distribution in training, which favors A); (b) factorial-style data augmentation. Pezeshkpour and Hruschka (2023) document the strong order-bias effect that this augmentation directly counters. Empirically, V_β (which bundles choice-permutation with α/r tuned from 2.0 down to 1.0) gains +3 pp on validation over the otherwise-identical lora-001 recipe; we cannot fully decompose the contributions of permutation alone vs. the alpha change, since the $\alpha/r = 2.0$ + permute cell of Table 4 was never run.

3.5 Retrieval Overlay

The dataset has substantial cross-split image overlap. We exploit this by computing a perceptual hash (Zauner, 2010) of every train, validation, and test image, then for each val/test row finding the train neighbor minimizing Hamming distance. We label a match as a *direct-copy* candidate when (i) $\text{Hamming} \leq h$, (ii) Ratcliff–Obershelp question-text similarity (Ratcliff and Metzner, 1988) via Python’s `difflib` is $\geq q$, and (iii) the train and target choice sets coincide as multisets. For direct-copy matches we substitute the train neighbor’s answer (remapped to the target row’s choice ordering); otherwise we keep the model prediction. We tune (h, q) on validation and validate gates: combined val accuracy must not regress vs. base, the direct-copy slice must clear 0.80, and the slice must beat the base on the same slice. The overlay is pure CPU work and is fully composable with calibration and majority vote. *Framing*: this is a deterministic post-processor over the provided training data, not a second model and not external data. It consults `train.csv` images and labels (both permitted training signal) at inference time and applies a hand-tuned heuristic; the model produces the prediction whenever no train neighbor passes the gates.

3.6 Train∪Validation Final-Fit

After hyperparameter selection settled on V_β and a higher-LR variant we call iter4-vd, we trained one additional adapter (**final-fit**) on the concatenation of train and validation (4,157 rows) for 2 epochs at $\text{lr} = 4 \times 10^{-4}$ with $\alpha/r = 1.0$ and choice-permutation. Local validation is leaky on this run since val is now in the training set, so we expected to judge it only by the procedure rather than by the inflated local number; the actual public-LB outcome (Section 6.1) was a regression, not a gain.

3.7 Inference Pipeline

Inference runs offline (HF_HUB_OFFLINE=1, TRANSFORMERS_OFFLINE=1, local_files_only=True). For each row we build the prompt deterministically, load and resize the image to 224×224 , run a single forward pass under bfloat16 autocast, gather logits at the last non-pad position, and argmax over candidate-letter ids. We dump per-row score matrices (val_scores.json, test_scores.json) for every full run so that calibration, retrieval, and majority-vote post-processors run as cheap CPU-only follow-ups.

4 Experimental Setup

4.1 Infrastructure

All training and full-validation evaluation ran on a single L40S GPU. A full V_β -style training run takes 195–210 minutes (5 epochs at 32 minutes train + 9 minutes eval per epoch). Inference loads the model in offline mode after a one-time snapshot download. Per-epoch resumable checkpoints (optimizer + scheduler + RNG + best-tracking) saved a lot of pain when long evaluation phases occasionally got killed for low GPU utilization.

4.2 Validation Protocol

We use the Kaggle-provided 1,048-row validation set as our hold-out for hyperparameter selection. Public-leaderboard accuracy on Kaggle is treated as a first-class signal that overrides local-val ranking when the two disagree (Section 6.1). For threshold sweeps over retrieval (h, q), we score the saved val_scores.json from each adapter, which lets us run $h \in \{2, 3, 4\} \times q \in \{0.85, 0.90\}$ in seconds without GPU.

4.3 Experiment Scope

We organize experiments into five rough phases. Each phase ends when one or two strong variants get promoted to a full-scale run; weaker variants are dropped. “Small-scale” here means a 16% train slice $\times$ 2 epochs $\times$ partial validation, used to triage a hyperparameter direction in roughly 25 minutes per run.

- **Setup:** model cache preflight, zero-shot baseline, always-A floor.
- **First LoRA:** $r=8$ / all-7 modules / $\alpha=16$ / $\text{lr}=2 \times 10^{-4}$ / no permute.
- **Architecture sweep:** connector unfreeze (V_α), the lora-001 recipe with $\alpha=8$ and choice-permutation (V_β), and attention-only at $r=16$ (V_γ). We use the Greek-letter labels ($V_\alpha \dots V_\varepsilon$) as shorthand for these named variants throughout the paper.
- **Hyperparameter sweep:** 8 small-scale variants over prompt content, image size, learning rate, weight decay, and dropout; full-scale LR variants V_δ ($\text{lr}=5\text{e-}5$) and V_ε ($\text{lr}=8\text{e-}4$); plus iter4-vd, our V_β recipe at $\text{lr}=4 \times 10^{-4}$ that became the base for the winning retrieval-overlay submission. Small-scale tests of chat-template prompting, native-resolution input, DoRA, and using solution as training context were also run here.
- **Post-processors:** retrieval overlay, per-letter calibration, train∪val final-fit, three-way majority vote.

5 Results

5.1 Overall Performance

Table 1 reports our submission ladder. **The best public-LB result is iter4-vd + retrieval at 0.861.** V_β plain at 0.86 is a close second — the gap is one test row out of 1,008, comfortably within sampling noise. Both submissions we ranked higher *a priori* (finfit, finfit + retrieval) scored lower on public LB than either of the top two.

5.2 Ablation Studies

We isolate the contribution of each design choice on validation accuracy. Each subsection below changes one axis with all others held at the V_β baseline.

5.2.1 LoRA Target-Module Set

All-7 modules wins decisively. Attention-only with $r=16$ spends *more* parameters and lands -9 pp be-

Pipeline	Val	Public LB
Always-A floor	0.331	—
Zero-shot	0.558	—
LoRA r=8 (lora-001)	0.7767	—
RunB native-res	0.7834	0.84
V_β	0.8073	0.86
iter4-vd	0.8225	0.84
iter4-vd + retrieval	0.8483	0.861
finfit (train $\cup$ val)	0.8674*	0.823
finfit + retrieval	0.8874*	0.849

Table 1: Submission ladder. Asterisks mark leaky validation (final-fit was trained on train $\cup$ val). The two finfit-based submissions underperformed V_β and iter4-vd + retrieval on public LB despite higher validation accuracy.

Target modules	Trainable	Val acc
attention-only (q/k/v/o), r=16	4.92 M	0.7195
all-7 (q/k/v/o + MLP), r=8 (V_β)	4.78 M	0.8073
all-7 + connector LoRA (V_α)	4.45 M	0.78 [†]

Table 2: Target-module ablation. [†]cancelled at epoch 2 after trailing V_β by ≈ 3 pp. All-7 dominates attention-only at this parameter budget.

low V_β , suggesting the MLP modules carry more task-specific signal than the attention modules at this scale.

5.2.2 Learning Rate

This is the result that drove most of our late-project decisions: **higher local validation accuracy was negatively correlated with public-leaderboard accuracy in the 4×10^{-4} to 8×10^{-4} range.** The iter4-vd adapter overfit to val features that did not transfer to the held-out public test. We come back to this in Section 6.1.

5.2.3 Choice-Permutation Augmentation

The two interventions were bundled in V_β , so we cannot fully separate them. [Pezeshkpour and Hruschka \(2023\)](#) report that order randomization alone is a strong intervention for instruction-tuned LLMs on MCQ tasks, which is consistent with our prior that most of the +3 pp gain is the permutation rather than the alpha change.

5.2.4 Retrieval Overlay Thresholds

Loosening the Hamming threshold from $h = 2$ to $h = 4$ adds 27 direct-copy candidates and +0.2 pp val with no observed gate violations. Hamming $h = 3$ adds nothing because the underlying distribution of cross-split duplicates clusters at $h \leq 2$ and $h \leq 4$.

Setting	Val	Public LB
lr= 5×10^{-5} + warmup 10% (V_δ)	0.79	—
lr= 2×10^{-4} (V_β)	0.8073	0.86
lr= 4×10^{-4} (iter4-vd)	0.8225	0.84
lr= 8×10^{-4} (V_ϵ)	0.7929	—

Table 3: Learning-rate sweep. Higher local val *does not* imply higher public LB: iter4-vd’s +1.5 pp local gain over V_β became −2 pp on public.

Setting	α/r	Val acc
no permute, $\alpha/r = 2.0$ (lora-001)	2.0	0.7767
permute, $\alpha/r = 2.0$	2.0	not run
permute, $\alpha/r = 1.0$ (V_β)	1.0	0.8073

Table 4: Choice-permutation + alpha-scaling ablation. The combined intervention adds +3.06 pp.

6 Analysis

6.1 Local Validation Overfits Public Leaderboard

The most surprising thing we saw: V_β at val 0.8073 scored **0.86** on public LB, while iter4-vd at val 0.8225 (+1.5 pp) scored only **0.84** (−2 pp). Retrieval applied on top of iter4-vd recovered it past V_β to **0.861**, our best result. The pattern got worse for our highest-confidence picks: finfit (train $\cup$ val merge) scored only 0.823 on public despite a leaky val of 0.867, and layering retrieval on top brought it to 0.849 — still below both V_β and iter4-vd + retrieval. The direction of the gap is consistent: more aggressive learning rates (lr= 4×10^{-4} , 8×10^{-4}) and training on the validation set found adapters that fit features specific to the validation split rather than features that generalized. With only 1,048 validation rows and a 5 M-parameter adapter, the val signal saturates at roughly ± 2 pp resolution and stops being a reliable selection signal between close variants.

6.2 Retrieval was the Highest-Leverage Move

The retrieval overlay added +2.6 pp val (and roughly +10 pp on the direct-copy slice) at zero GPU cost, and on top of iter4-vd it transferred cleanly to +2.1 pp on public LB — enough to make iter4-vd + retrieval our best submission at 0.861. On top of finfit, however, retrieval still added validation accuracy but the underlying base was already overfitted to val, and retrieval did not rescue it on public. By contrast, the small-scale hyperparameter sweep and the full-scale LR variants together consumed roughly 10–15 GPU-hours and moved

h	q	Val	Slice	Overrides
base (iter4-vd)	—	0.8225	—	0
2	0.85	0.8464	240	41
3	0.85	0.8464	240	41
4	0.85	0.8483	267	49
4	0.90	0.8473	252	45

Table 5: Retrieval threshold sweep over saved val scores. **Slice** is the count of validation rows that pass direct-copy gates. **Overrides** is the count of test rows where retrieval substitutes the model prediction.

validation by 0 to +3 pp at best. With hindsight the right ordering would have been: train one strong-enough LoRA early, then spend remaining compute on post-processors (retrieval, calibration, ensembling) rather than on more hyperparameter tuning. See Section 7.

6.3 What Did Not Work

Chat-template prompting via `processor.apply_chat_template` was a strongly-recommended fix in related work but landed at val 0.675 on a small-scale run vs. 0.7075 for the manual prompt — no improvement. The prompt format was not the bottleneck. **Native-resolution input** (`do_image_splitting`) beat pre-resize on a small-scale run (+0.7 pp val) but *lost* on public LB (0.84 vs. V_β 's 0.86). **DoRA + rsLoRA** (Liu et al., 2024; Kalajdzievski, 2023) tripped the 5 M cap at $r = 8$; at $r = 7$ it OOM'd at the first batch. **Solution-as-context** during training regressed -5 pp at small scale, plausibly because the solution column is in train but not test, so the model learns to lean on a feature that disappears at inference.

7 Limitations

- **We did not hit the 0.90 target.** The public-LB ceiling at the deadline was 0.861 (iter4-vd + retrieval). The submissions we expected to do better — finfit and finfit + retrieval — scored *worse* on public (0.823 and 0.849). The goal was 0.90 absolute, and we fell short.
- **Local validation overfits the public leaderboard.** 1,048 val rows is not enough resolution to discriminate adapters within ± 2 pp of each other, and once we started training on val (finfit) the local signal became actively misleading rather than just noisy. With another day the right move would have been to hold out a second validation slice stratified from

train and never let it leak into training.

- **Retrieval was identified late.** A retrieval-style cross-split exploit was visible from inspecting the data on day 1, but we only built it into the experiment loop after most of the training compute had been spent. If retrieval had come first, the project would have spent more compute on the post-processor stack and less on hyperparameter tuning.
- **The 5 M parameter cap limited adapter-scheme search.** DoRA (Liu et al., 2024) has been reported to outperform vanilla LoRA on related benchmarks, but our DoRA + rsLoRA variant tripped the 5 M cap at $r = 8$ and OOM'd at the first batch at $r = 7$, so we could not evaluate it under the project's parameter budget.
- **Ablations are not strictly factorial.** V_β bundles two changes (alpha/r and permutation), so we cannot fully separate their individual contributions.

8 Conclusion and Future Directions

We trained SmolVLM-500M-Instruct with LoRA on a multimodal scientific multiple-choice task under a 5 M trainable-parameter cap. Our top public-LB result, 0.861, comes from a higher-LR LoRA variant (iter4-vd, $lr=4 \times 10^{-4}$, all-7 / $\alpha/r = 1.0$ / choice-permutation) with a perceptual-hash retrieval overlay applied at $h = 4$, $q = 0.85$. Plain V_β is a close second at 0.86, within sampling noise of the top result; without the retrieval overlay, iter4-vd alone drops to 0.84. Across twelve ablations the most consistent takeaway is that cheap CPU-side post-processors (retrieval, calibration) outperformed incremental hyperparameter sweeps once a strong-enough adapter existed — but only when the underlying base was not itself overfit to validation, as the finfit-based submissions demonstrated.

The clearest next step is a multi-seed ensemble: average logits across two or three V_β -style adapters before the choice-letter argmax. We expect +1 to +2 pp on a different axis from retrieval. Test-time augmentation via choice permutation — scoring each test row under N random orderings of choices and averaging per-choice scores — would also be cheap, addresses position bias more directly than the val-fitted bias term in calibration, and would have been faster to ship than another LR sweep.

The bigger lesson from this project, for compact-VLM multiple-choice work specifically: the bot-

tleneck moves quickly from the model to the data. Looking for cross-split exploits like retrieval on shared images, and dumping per-row scores so calibration and ensembling are cheap follow-ups, are higher-leverage than another round of LR / dropout / weight-decay tuning once the adapter is strong enough.

Reproducibility

All training and inference code is at <https://github.com/adamfweidman/deep-learning-final-submission-repo>.

The repository contains the per-experiment YAML configs, batch scripts for the L40S training runs, an inference-only Jupyter notebook, and a pinned requirements.txt. The iter4-vd adapter (base for the winning 0.861 submission) is hosted at <https://drive.google.com/file/d/1H9PaPqCkqgFRtIRiNhHy4e51f58VGJ63/view?usp=sharing>. Random seeds are fixed in one place (src.run._seed_all) covering Python random, NumPy, PyTorch CPU & CUDA, transformers.set_seed, PYTHONHASHSEED, and cuDNN deterministic mode; per-epoch torch.Generator keeps the DataLoader shuffle order reproducible.

AI Tooling Disclosure

Claude Code (Opus) was used as a coding/debugging assistant throughout the project: scaffolding the runnable Python surface from the starter notebook, drafting per-experiment YAML configs, debugging tokenizer-fork deadlocks, and co-writing this report. Experimental design decisions (which ablations to run, which thresholds to sweep, which submissions to declare final) were made by the author. All numerical results in tables and figures were produced by jobs whose configs are checked into the repository.

Acknowledgments

We thank the course staff for the assignment infrastructure.

References

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shengyu Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Damjan Kalajdzievski. 2023. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*.

Hugo Laurençon, Andrés Maráfioti, Victor Sanh, and Léo Tronchon. 2024. Building and better understanding vision-language models: Insights and future directions. *arXiv preprint arXiv:2408.12637*.

Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. DoRA: Weight-decomposed low-rank adaptation. *arXiv preprint arXiv:2402.09353*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. *International Conference on Learning Representations*.

Pouya Pezeshkpour and Estevam Hruschka. 2023. Large language models sensitivity to the order of options in multiple-choice questions. *arXiv preprint arXiv:2308.11483*.

John W Ratcliff and David E Metzener. 1988. Pattern matching: The gestalt approach. *Dr. Dobb's Journal*.

Polina Tsvilodub, Hening Wang, Niklas Grosse, Marius Mosbach, Vera Demberg, and Michael Franke. 2024. Predictions from language models for multiple-choice tasks are not robust under variation of scoring methods. *arXiv preprint arXiv:2403.00998*.

Sarah Wiegrefe, Matthew Finlayson, Oyvind Tafjord, Peter Clark, and Ashish Sabharwal. 2023. Increasing probability mass on answer choices does not always improve accuracy. *arXiv preprint arXiv:2305.14596*.

Christoph Zauner. 2010. Implementation and benchmarking of perceptual image hash functions. *Master's Thesis, University of Applied Sciences Hagenberg*.

A Full Experiment Log

Table 6 lists every full-scale run that produced a recorded validation accuracy. Small-scale runs (16% train $\times$ 2 epochs $\times$ partial val), used for triaging hyperparameter directions, are omitted from this table.

B Training Data Statistics

- Train: 3,109 rows / 3,109 images.
- Validation: 1,048 rows / 1,048 images.
- Test: 1,008 rows / 1,008 images.
- Number of choices per row: 2–5; the modal value is 4. Only 38 of 1,008 test rows have 5 choices, which is why our predictions cover 0–3 for 99.6% of rows.

Phase	Run	Trainable	lr	α/r	Val acc	Public LB
<i>Phase 0: Baselines</i>						
0a	always-A	0	—	—	0.331	—
0b	zero-shot	0	—	—	0.558	—
<i>Phase 1: First LoRA</i>						
1	lora-001 (r=8 / all-7 / $\alpha=16$)	4.78 M	2e-4	2.0	0.7767	—
<i>Phase 2: Iter-3 Architecture Bundle</i>						
2a	V_α (connector LoRA)	4.45 M	2e-4	1.0	0.78 [†]	—
2b	V_β (all-7 + $\alpha/r = 1.0$ + permute)	4.78 M	2e-4	1.0	0.8073	0.86
2c	V_γ (attn-only r=16)	4.92 M	2e-4	1.0	0.7195	—
<i>Phase 3: Iter-4 LR + Architecture Sweeps</i>						
3a	RunB native-res	4.78 M	2e-4	1.0	0.7834	0.84
3b	iter4-vd (lr=4e-4)	4.78 M	4e-4	1.0	0.8225	0.84
3c	V_ϵ (lr=8e-4)	4.78 M	8e-4	1.0	0.7929	—
3d	iter4-vd replicate, seed 43	4.78 M	4e-4	1.0	0.81	—
<i>Phase 4: Iter-5 Post-Processors</i>						
4a	calibration-vd (per-letter bias)	0	—	—	0.8292	—
4b	retrieval overlay on iter4-vd ($h=4, q=0.85$)	0	—	—	0.8483	0.861
4c	retrieval overlay on V_β ($h=4$)	0	—	—	not run on val	not submitted
4d	finfit (train $\cup$ val, lr=4e-4)	4.78 M	4e-4	1.0	0.8674*	0.823
4e	retrieval-finfit-h4 (overlay on finfit)	0	—	—	0.8874*	0.849

Table 6: Full-scale run log. *denotes leaky validation (training included val). [†]denotes a run cancelled before completion. Trainable counts include LoRA weights; the base 502.7 M frozen parameters are excluded.

- Cross-split image overlap: $\approx 26\%$ of validation rows have a perceptual-hash neighbor in train at $h \leq 4$ with question similarity ≥ 0.85 and matching choice set; $\approx 26\%$ of test rows have the same property.
- Ground-truth label distribution on validation: roughly $\{0 : 33\%, 1 : 27\%, 2 : 26\%, 3 : 12\%, 4 : 2\%\}$ (mild A-bias).